java production issues

06/04/2026 19:33:19
https://claude.ai/chat/42600ca7-45b3-4031-bbb0-9879a6ad3214

Java springboot microservices production q&a and how to fix that and how we would have avoided earlier

6/2/2026, 5:38:48 PM

I'll create a comprehensive interactive Q&A app for Java Spring Boot microservices production issues.

Viewed file

Check frontend design skill

Done

6/2/2026, 5:39:11 PM

Don't create any interactive design or documents write it here at least 50 q&a I want to see

6/2/2026, 5:39:49 PM

Here are 50 Java Spring Boot Microservices Production Q&A covering real issues, fixes, and prevention strategies.

---

1. What causes `OutOfMemoryError: Java heap space` in production and how do you fix it?

Cause: Large object allocations, memory leaks (unclosed streams, static collections growing unbounded), or heap size too small for load.

Fix: Increase heap with `-Xmx` and `-Xms` JVM flags. Take a heap dump using `jmap -dump:format=b,file=heap.hprof <pid>` and analyze with Eclipse MAT or VisualVM. Find the leak — commonly large lists cached without eviction, Hibernate session objects not closing, or Kafka consumer buffers piling up.

Prevention: Set explicit heap limits in Kubernetes resource requests/limits. Use weak references for caches. Add Micrometer JVM memory metrics and alert when heap usage exceeds 80%.

---

2. Circuit breaker is open but downstream service has recovered — requests are still failing. Why?

Cause: Resilience4j's `waitDurationInOpenState` hasn't elapsed, so the circuit stays open even after the downstream recovers.

Fix: Tune `waitDurationInOpenState` (default is 60s). You can also expose a management endpoint to manually reset the circuit: `POST /actuator/circuitbreakers/{name}/reset` if you've enabled it.

Prevention: Set `waitDurationInOpenState` based on your SLA — not too long. Add metrics and alerting on circuit breaker state transitions so you know when it opens and when it closes.

3. Spring Boot app starts fine locally but crashes in Kubernetes with `CrashLoopBackOff`. What do you check?

Cause: Usually one of — missing environment variables, wrong database URL, readiness probe failing before app is ready, or insufficient memory causing OOM kill.

Fix: Run `kubectl logs <pod> --previous` for the crash log. Check `kubectl describe pod <pod>` for OOM kills. Validate all env vars are injected from ConfigMap/Secret. Check if the liveness probe is too aggressive and killing the app during startup.

Prevention: Use `startupProbe` to give the app enough time to initialize. Set resource requests/limits correctly. Use Spring's `@ConfigurationProperties` with validation so missing config fails fast at startup.

---

4. Kafka consumer lag keeps growing. How do you diagnose and fix it?

Cause: Consumer is too slow — either processing is CPU-heavy, downstream DB calls are slow, or there aren't enough partitions/consumers to parallelize.

Fix: Check lag with `kafka-consumer-groups.sh --describe`. Increase `concurrency` in `@KafkaListener`. If DB is the bottleneck, batch-insert instead of row-by-row. If CPU-heavy, move processing to a thread pool with `@Async`. Scale consumer instances up to match partition count.

Prevention: Always size partitions ahead of expected TPS. Monitor consumer lag via Kafka JMX metrics or Confluent/MSK dashboards. Set a lag threshold alert — don't wait for users to report delays.

---

5. `@Transactional` is not rolling back on a checked exception. Why?

Cause: By default, Spring only rolls back on unchecked exceptions (`RuntimeException` and `Error`). Checked exceptions are not rolled back unless you explicitly configure it.

Fix: Use `@Transactional(rollbackFor = Exception.class)` or convert the checked exception to a `RuntimeException`.

Prevention: Establish a team convention — either always use `rollbackFor = Exception.class` or always throw `RuntimeException` subclasses from service layers. Document this in your coding standards.

---

6. REST API response time spikes every few minutes. What's the likely cause?

Cause: JVM garbage collection pauses (G1GC or ZGC stop-the-world events), connection pool exhaustion, or a scheduled job running at that interval competing for resources.

Fix: Enable GC logging: `-Xlog:gc*` and analyze with GCEasy. If it's a GC issue, tune heap regions or switch to ZGC. If it's connection pool, increase `maximum-pool-size` in HikariCP. Check `@Scheduled` jobs and move them off the main thread pool.

Prevention: Add p99 latency metrics. Set GC pause time alerting. Review all `@Scheduled` tasks for resource contention at design time.

---

7. Database connection pool exhausted — `HikariPool-1 - Connection is not available, request timed out after 30000ms`. How do you fix it?

Cause: Connections are being held too long (slow queries, missing transaction close, N+1 queries), or pool size is too small for concurrent load.

Fix: Check slow queries via `SHOW PROCESSLIST` in MySQL or `pg_stat_activity` in Postgres. Enable HikariCP metrics. Set `connection-timeout`, `idle-timeout`, and `max-lifetime` appropriately. Increase `maximum-pool-size` if load genuinely warrants it.

Prevention: Size the pool based on `number of cores * 2 + disk spindles` (HikariCP recommendation). Use `@Transactional(readOnly = true)` for reads. Always close resources in finally blocks. Enable slow query logs.

---

8. A Spring Boot service is leaking file descriptors. How do you detect and fix it?

Cause: Unclosed `InputStream`, `RestTemplate` connections not released, or Kafka consumer not closed properly on error.

Fix: Check with `lsof -p <pid> | wc -l` or via `/proc/<pid>/fd`. Analyze which file type (sockets, files) is leaking. Add `try-with-resources` everywhere. For `RestTemplate`, switch to `WebClient` with connection pooling configured. For Kafka, ensure consumers are closed in a `@PreDestroy` method.

Prevention: Use `try-with-resources` as a team standard. Add file descriptor metrics via Micrometer. Alert when `open_file_descriptors` approaches the system limit (`ulimit -n`).

9. Feign client throws `RetryableException` randomly in production. What do you investigate?

Cause: Network flakiness, target service is briefly unavailable, or DNS resolution is failing intermittently — especially in Kubernetes where pod IPs change.

Fix: Add a Feign retryer with backoff. Check if the target service is restarting frequently (rolling deploy?). If DNS is the issue, use service names instead of IPs and ensure CoreDNS is healthy. Log the exact exception and target URL on every failure.

Prevention: Pair Feign with Resilience4j circuit breaker. Set appropriate `connectTimeout` and `readTimeout`. Use service mesh (Istio) for transparent retries at the infrastructure level.

---

10. `LazyInitializationException` in production but not in development. Why?

Cause: In production, the Hibernate session closes before lazy-loaded collections are accessed — typically because the transaction boundary ends in the service layer but the serialization happens in the controller.

Fix: Either fetch eagerly using `JOIN FETCH` in JPQL, use a DTO projection, or use `@Transactional` at the controller level (not recommended). Best option: use `EntityGraph` or a DTO mapper like MapStruct that forces loading within the transaction.

Prevention: Disable `spring.jpa.open-in-view=true` (it's enabled by default and hides this problem in dev). With it disabled, you'll catch this issue in integration tests, not production.

---

11. Kafka consumer is processing duplicate messages. How do you make it idempotent?

Cause: Consumer processed the message but crashed before committing the offset, so on restart it re-reads the same message.

Fix: Design consumers to be idempotent — check if the entity already exists before inserting. Use a `processed_message_id` table to track already-handled message IDs. Use database unique constraints as a safety net.

Prevention: Always design Kafka consumers with at-least-once delivery in mind. Use `enable.auto.commit=false` and commit offsets only after successful processing. Add

message deduplication at the business logic layer.

---

12. Spring Security is blocking inter-service calls in production but not locally. Why?

Cause: Services have security filters applied that require JWT tokens. Local `dev` profile might bypass security, but production profile enforces it.

Fix: Pass the JWT token in inter-service calls via Feign request interceptor: implement `RequestInterceptor` to extract the current SecurityContext token and add it to outgoing headers. For service-to-service (machine-to-machine), use client credentials OAuth2 flow.

Prevention: Test with security enabled in CI. Use Spring Security test annotations in integration tests. Maintain separate OAuth2 client credentials for service accounts.

---

13. Service mesh shows high error rate but Spring Boot actuator health is `UP`. How?

Cause: Health endpoint only checks the service itself (DB connection, disk). It doesn't know about upstream callers failing due to schema mismatches, incompatible API versions, or serialization errors.

Fix: Add custom health indicators that probe actual business logic paths. Use distributed tracing (OpenTelemetry/Jaeger) to trace where errors originate. Check Istio/Envoy access logs for 4xx/5xx breakdown.

Prevention: Add composite health checks that include downstream dependencies. Use contract testing (Pact) to catch API incompatibilities before deploy. Monitor business-level error rates, not just infrastructure health.

---

14. Redis cache is causing stale data issues in production. How do you handle it?

Cause: TTL is too long, or cache is not being evicted when the underlying data changes.

Fix: Use `@CacheEvict` on write methods. For distributed cache invalidation across services, publish a cache-invalidation event on Kafka and have all instances evict on receipt. Reduce TTL as a short-term fix.

Prevention: Apply Cache-Aside pattern explicitly rather than relying on `@Cacheable` blindly. Define TTL per cache region based on data volatility. Test cache invalidation in integration tests.

15. Aurora MySQL reader endpoint is getting write traffic in production. What happened?

Cause: `@Transactional(readOnly = true)` is not set on read methods, so the routing datasource sends all traffic to the writer. Or the AbstractRoutingDataSource lookup key is not set correctly in ThreadLocal.

Fix: Ensure all read-only service methods are annotated with `@Transactional(readOnly = true)`. Verify the `AbstractRoutingDataSource.determineCurrentLookupKey()` method reads the transaction's readOnly flag correctly.

Prevention: Use a custom AOP aspect to validate that all `find*` / `get*` / `list*` methods have `readOnly = true`. Add monitoring on reader vs writer query counts via CloudWatch/RDS metrics.

---

16. The application is running but a new feature's API returns 404. Spring MVC controller is not mapping. Why?

Cause: Controller class is not in a package scanned by Spring — either placed outside the base package or in a module not included in component scan. Or there's a context hierarchy issue in a multi-module project.

Fix: Check `@SpringBootApplication` base package. Move the controller to the scanned package or add it explicitly with `@ComponentScan`. Verify the module is on the classpath.

Prevention: Use a consistent package structure convention. Write a smoke test for every new endpoint as part of CI.

---

17. JWT token validation is working but authorization fails for some users. What do you check?

Cause: Role/authority mapping from the JWT claims is incorrect — either the claim name doesn't match what Spring Security expects, or roles are missing the `ROLE_` prefix.

Fix: Debug the `Authentication` object in a filter: log `SecurityContextHolder.getContext().getAuthentication().getAuthorities()`. Check the JWT converter — if using `JwtGrantedAuthoritiesConverter`, ensure the claim name and prefix match your token structure.

Prevention: Write parameterized unit tests for the JWT converter with various token shapes. Use a dedicated auth integration test suite that covers edge cases like missing

roles, expired tokens, and malformed claims.

---

18. Scheduled job runs multiple times when multiple instances are deployed. How do you prevent this?

Cause: Each instance has its own `@Scheduled` executor and runs the job independently — there's no distributed coordination.

Fix: Use ShedLock with a shared store (Redis or DB) to ensure only one instance runs at a time. Annotate the method with `@SchedulerLock(name = "jobName", lockAtLeastFor = "PT10M")`.

Prevention: Default to ShedLock from the first day a scheduled job is added. Never assume a scheduled job is single-instance safe in a horizontally scaled environment.

---

19. Distributed transaction across two microservices failed halfway. One service committed, the other didn't. How do you recover?

Cause: Two-phase commit (2PC) is not used — each service has its own local transaction. A network timeout or crash between the two calls left the system in an inconsistent state.

Fix: Implement the Saga pattern. Use either choreography (Kafka events) or orchestration (a saga orchestrator service). For the failed saga, replay the compensating transaction — e.g., if order was created but payment failed, emit an `OrderCancelled` event.

Prevention: Design all cross-service operations as sagas from the start. Never use synchronous dual writes across services. Use an outbox table to ensure event publishing is atomic with the local DB commit.

---

20. Spring Boot app takes 90 seconds to start in production. How do you speed it up?

Cause: Large number of beans, slow database migrations (Flyway/Liquibase), blocking I/O during startup, or classpath scanning over too many packages.

Fix: Use Spring Boot's lazy initialization: `spring.main.lazy-initialization=true`. Run Flyway migrations as a separate init container in Kubernetes. Enable Spring AOT compilation (Spring Boot 3+). Profile startup with `spring.jvm.checkpoint.restore` or `-Dspring.context.verbose=true`.

Prevention: Set a startup time SLA. Measure startup time in CI and alert on regression. Separate migration from app startup.

21. `ConcurrentModificationException` thrown under load. How do you diagnose it?

Cause: A `HashMap` or `ArrayList` is shared across threads without synchronization, and one thread modifies the collection while another iterates it.

Fix: Replace with `ConcurrentHashMap` or `CopyOnWriteArrayList`. If it's a Spring bean (singleton), make the field immutable or use `Collections.synchronizedList()`. Take a thread dump during the issue with `jstack <pid>` to pinpoint the class.

Prevention: Never use mutable instance variables in singleton Spring beans. Use thread-local state or pass data as method parameters. Enable lock-order analysis in code review.

---

22. Database deadlock detected in production — `Deadlock found when trying to get lock; try restarting transaction`. How do you handle it?

Cause: Two transactions are waiting for each other's locks — typically caused by inconsistent lock ordering (Transaction A locks row 1 then row 2, Transaction B locks row 2 then row 1).

Fix: Add retry logic using Spring Retry: `@Retryable` on the method with `include = DeadlockLoserDataAccessException.class`. Fix lock ordering at the application level — always acquire locks in the same order. Add `SELECT ... FOR UPDATE` explicitly where needed.

Prevention: Use optimistic locking (`@Version`) for high-contention entities. Reduce transaction scope. Analyze deadlock logs in MySQL (`SHOW ENGINE INNODB STATUS`) and fix the lock ordering.

---

23. API Gateway (Tyk) is returning 502 Bad Gateway intermittently. What do you check?

Cause: Upstream service is slow or crashing, Tyk's upstream timeout is too low, or the upstream target URL is stale (pointing to a dead pod IP).

Fix: Check Tyk upstream timeout settings. Verify target URLs use Kubernetes service DNS names, not pod IPs. Check upstream service health and logs. Look at Tyk's analytics dashboard for error rate patterns.

Prevention: Always use service DNS names in Tyk API definitions. Set upstream timeouts to match your service's p99 response time. Use Tyk's circuit breaker at the gateway level as an additional safety net.

---

24. `StackOverflowError` in production for a specific request. What causes it?

Cause: Infinite recursion — a method calls itself directly or indirectly without a proper base case. Common in bidirectional JPA relationships causing Jackson serialization recursion.

Fix: For JPA/Jackson: use `@JsonManagedReference` / `@JsonBackReference` or `@JsonIgnore` on the back side of the relationship. For logical recursion: add null/base-case checks and a recursion depth guard.

Prevention: Ban bidirectional Jackson serialization of JPA entities. Use DTOs for API responses. Add integration tests for entities with relationships.

---

25. Health check endpoint `/actuator/health` is exposed to the internet. What's the risk and fix?

Cause: Spring Boot Actuator endpoints are not properly secured — they're accessible on the same port as the API without authentication.

Fix: Run Actuator on a separate management port (`management.server.port=8081`) and restrict it to internal traffic via VPC/security group rules. Or use Spring Security to require authentication for actuator endpoints.

Prevention: Security-scan actuator exposure in every deployment pipeline. Use a network policy in Kubernetes to block external access to the management port. Default to securing all actuator endpoints in your base `SecurityConfig`.

---

26. Service is returning stale Feign client response after a downstream update. Why?

Cause: Feign client response is being cached (via Spring Cache or an HTTP cache header being honored) and not refreshed after the downstream service updates.

Fix: Check if `@Cacheable` is applied on the Feign client method. Remove it or reduce TTL. Check HTTP response headers for `Cache-Control` headers. Use `@CacheEvict` when data is known to change.

Prevention: Don't apply `@Cacheable` directly on Feign clients without explicit eviction strategy. Treat inter-service calls as live network calls unless caching is intentional and tested.

---

27. Kafka producer is losing messages silently. How do you ensure delivery?

Cause: `acks=0` or `acks=1` configuration means the broker doesn't wait for replication before acknowledging. App crashes after send but before replication completes.

Fix: Set `acks=all` (or `acks=-1`). Set `retries=Integer.MAX_VALUE` and `enable.idempotence=true`. Use `KafkaTemplate.send().get()` to block and confirm delivery, or handle the `ListenableFuture` callback for async confirmation.

Prevention: Always use `acks=all` and idempotent producer in production. Test producer failure scenarios in staging. Log and alert on producer send failures.

---

28. New microservice deployment causes cascading failures across the system. How do you design against this?

Cause: Tight synchronous coupling — services call each other in a chain. One slow or failed service blocks threads upstream, causing timeout cascades.

Fix: Introduce Resilience4j bulkhead (thread pool isolation) so a failing downstream doesn't exhaust the caller's thread pool. Add timeouts on all Feign/WebClient calls. Use async/event-driven communication where possible.

Prevention: Apply bulkhead pattern from day one of service design. Define and enforce timeout budgets for every inter-service call. Use chaos engineering (kill a pod, inject latency) to validate resilience before going live.

---

29. Spring Boot app memory usage keeps growing after load testing. It's not a heap leak — what else could it be?

Cause: Metaspace leak (dynamic class generation — reflection, CGLIB proxies, Groovy scripts), direct memory leak (Netty buffers in reactive stacks), or native memory used by threads.

Fix: Add `-XX:MaxMetaspaceSize=256m` and monitor metaspace via Micrometer. For Netty direct memory: add `-Dio.netty.leakDetection.level=PARANOID` in staging to detect buffer leaks. Check thread count growth — too many threads consume stack memory.

Prevention: Profile memory holistically — heap + metaspace + direct + stack. Use NativeMemoryTracking: `-XX:NativeMemoryTracking=summary` and check via `jcmd <pid> VM.native_memory`.

---

30. How do you handle secret rotation (DB password, API keys) without restarting the service?

Cause: Secrets are injected as environment variables at startup and never refreshed. When rotated, the service still uses the old credentials until restart.

Fix: Use AWS Secrets Manager with Spring Cloud AWS. Configure the secret to refresh at a set interval. For DB passwords, use dynamic secrets with Vault — the datasource can be refreshed via `@RefreshScope` on the datasource bean.

Prevention: Never hardcode secrets or inject them only at startup without a refresh path. Prefer Vault or Secrets Manager with TTL-based rotation. Test secret rotation in staging before enabling in production.

---

31. `@Async` methods are not executing asynchronously — still running on the main thread. Why?

Cause: `@EnableAsync` is missing on the configuration class. Or the `@Async` method is called from within the same bean (self-invocation bypasses the Spring proxy).

Fix: Add `@EnableAsync` to your `@Configuration` class. Move the `@Async` method to a separate Spring bean so the call goes through the proxy.

Prevention: Write a test that verifies the method runs on a different thread. Establish a code review rule: never self-invoke `@Async` or `@Transactional` methods.

---

32. Production logs are flooded with noise — hard to find real errors. How do you fix this?

Cause: DEBUG or INFO log level is set globally, catching framework internals. No structured logging. No correlation IDs to filter by request.

Fix: Set root log level to WARN and only DEBUG your own packages. Switch to structured JSON logging with Logback + `logstash-logback-encoder`. Add MDC correlation ID (from trace ID or request header) to every log line. Ship to ELK or CloudWatch Logs Insights with filter queries.

Prevention: Mandate structured logging and correlation IDs in the service template. Review log output as part of every PR. Set log volume alerts — if log rate spikes unexpectedly, it's a signal.

---

33. Service responds with 200 OK but business logic silently failed. How do you catch this?

Cause: Exceptions are caught and swallowed somewhere — usually in a generic `catch (Exception e) { log.error(); }` block — and the method returns a default/empty value.

Fix: Audit all broad `catch` blocks. Never swallow exceptions without re-throwing or returning a meaningful error. Use a global `@ControllerAdvice` with proper exception-to-HTTP-status mapping. Add business-level metrics (e.g., `claims.processing.failed` counter) to detect silent failures.

Prevention: Ban bare `catch (Exception e)` in code review. Use a linter rule (Checkstyle/PMD) to flag swallowed exceptions. Add assertion-based integration tests that verify the actual DB state, not just the response code.

---

34. Spring Boot service is slow when connecting to Aurora — works fine with RDS. Why?

Cause: Aurora endpoints involve DNS TTL — Aurora's reader endpoint DNS TTL is short but Java's JVM DNS cache (`networkaddress.cache.ttl`) is set to forever by default in some environments. Or Aurora's connection overhead for its proxy/router layer adds latency.

Fix: Set `networkaddress.cache.ttl=60` in `java.security` or via `-Dsun.net.inetaddr.ttl=60`. Use RDS Proxy in front of Aurora to reduce connection overhead.

Prevention: Always configure JVM DNS TTL when using Aurora or any DNS-based failover. Test failover behavior in staging — Aurora failover changes the DNS target and JVM needs to re-resolve.

---

35. How do you safely do a zero-downtime database schema migration in production?

Cause: Running `ALTER TABLE` directly causes table locks in MySQL, blocking reads/writes for the duration. Running Flyway at startup causes a race condition if multiple instances start simultaneously.

Fix: Use `gh-ost` or `pt-online-schema-change` for large table migrations in MySQL — they work without table locks. For Flyway, use a Kubernetes init container to run migrations before the app pods start. Add the Flyway lock feature to prevent concurrent migrations.

Prevention: Test all migrations on a production-sized dataset in staging. Never do `ALTER TABLE` directly on large tables in production. Treat schema migration as a separate deployment step from app deployment.

---

36. WebClient (reactive) is blocking the event loop. Application hangs under load. Why?

Cause: A blocking call (`Thread.sleep`, JDBC, blocking I/O) is made inside a reactive pipeline — this blocks the Netty event loop thread, starving all other reactive tasks.

Fix: Use `Schedulers.boundedElastic()` to offload blocking operations: `Mono.fromCallable(() -> blockingCall()).subscribeOn(Schedulers.boundedElastic())`. Never use blocking APIs directly in `flatMap` or `map` in a reactive chain.

Prevention: Enable BlockHound in tests to detect blocking calls in reactive code. Establish a team rule: if you use WebClient, you cannot mix blocking JDBC unless you explicitly offload with `boundedElastic`.

---

37. Distributed trace spans are broken — trace IDs don't propagate across services. How do you fix it?

Cause: The `traceparent` / `X-B3-TraceId` header is not being forwarded in Feign or Kafka. Micrometer Tracing / Sleuth isn't configured to propagate headers automatically.

Fix: Add `micrometer-tracing-bridge-otel` and `opentelemetry-exporter-otlp` dependencies. For Feign, Sleuth auto-configures header propagation. For Kafka, add `KafkaTracingCustomizer` or configure `spring.kafka.producer.properties` with trace propagation.

Prevention: Validate end-to-end trace propagation in integration tests using Zipkin/Jaeger in your test environment. Test both HTTP and Kafka paths.

---

38. Feign client is timing out but the downstream service logs show the response was sent in <100ms. Why?

Cause: The timeout is occurring in the network/TCP layer between the two, not in the downstream business logic. Could be DNS resolution time, connection establishment to a cold connection pool, or Kubernetes network policy overhead.

Fix: Differentiate `connectTimeout` (how long to wait to establish TCP connection) from `readTimeout` (how long to wait for the response). Increase `connectTimeout` if connection setup is slow. Use connection pooling (Apache HttpClient with connection reuse) instead of default per-request connections.

Prevention: Always configure both `connectTimeout` and `readTimeout` explicitly. Use `Apache HttpClient` or `OkHttp` as the Feign HTTP client instead of the default `java.net.HttpURLConnection`.

---

39. `@Value` injection returns null in a utility/helper class. Why?

Cause: The class is instantiated with `new MyHelper()` instead of being a Spring-managed bean. Spring can only inject `@Value` into beans it manages.

Fix: Annotate the class with `@Component` and inject it via `@Autowired` or constructor injection. Never use `new` to instantiate classes that need Spring injection.

Prevention: Enforce constructor injection over field injection — it makes instantiation issues visible immediately (NullPointerException at construction time, not runtime). Use `@RequiredArgsConstructor` (Lombok) as the standard.

---

40. API is returning 500 on large payloads but works fine for small ones. What do you check?

Cause: `spring.servlet.multipart.max-file-size` or `spring.servlet.multipart.max-request-size` limit is breached. Or Jackson's `DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES` + large payload causes OOM during parse.

Fix: Increase limits in `application.yml`. If payloads are genuinely large, stream them with `InputStream` instead of loading entirely into memory. Use Jackson's `@JsonIgnoreProperties` to avoid parsing unnecessary fields.

Prevention: Define a max payload size policy for every endpoint. Document it in the API contract. Test with max-size payloads in integration tests.

---

41. Rate limiter in Tyk is rejecting legitimate requests. How do you tune it?

Cause: Rate limit is set per API key globally but the system handles burst traffic (batch processing, end-of-month loads) that legitimately exceeds the steady-state rate.

Fix: Use token bucket algorithm (Tyk default) which allows bursting above the sustained rate. Increase the burst multiplier. For batch clients, issue a separate API key with a higher limit. Add `Retry-After` headers so clients back off gracefully.

Prevention: Set rate limits based on load test data, not guesses. Model burst patterns (daily peaks, batch jobs) and size limits accordingly. Monitor `429` rates in Tyk analytics.

---

42. CQRS read model (MongoDB) is out of sync with the write model (Aurora MySQL). How do you detect and fix it?

Cause: The Kafka event that should have updated the read model was lost, failed during processing, or was processed out of order.

Fix: Add a reconciliation job that periodically compares counts/checksums between write and read models. For the missed event — replay from Kafka using `--from-beginning` on the affected consumer group for the affected time window. Add dead letter queue (DLQ) handling for failed read-model update events.

Prevention: Add DLQ for every Kafka consumer that updates the read model. Implement idempotent event handlers. Build a reconciliation dashboard as a health indicator.

---

43. Spring Boot app is crashing with `Address already in use: bind` on port 8080. What happened?

Cause: A previous instance of the app didn't shut down cleanly and the socket is still bound. Or a different process on the same host is using port 8080.

Fix: `lsof -ti:8080 | xargs kill -9` to free the port. In Kubernetes, this usually means the old pod is in `Terminating` state with a finalizer blocking deletion. Force-delete: `kubectl delete pod <pod> --force --grace-period=0`.

Prevention: In Kubernetes, use `preStop` lifecycle hooks and `terminationGracePeriodSeconds` to give pods enough time to drain before force-termination. Set proper shutdown hooks in Spring Boot with `server.shutdown=graceful`.

---

44. How do you handle a bad Flyway migration that's already run in production?

Cause: A migration script had a bug — wrong column name, bad data transform — and it ran against production before the issue was found.

Fix: Never edit a migration that has already run. Create a new migration script that corrects the damage. If data was corrupted, write a compensating migration. If the schema is wrong, write an `ALTER TABLE` migration to fix it. Update the Flyway checksum with `flyway repair` if you must patch the script in an emergency and the migration hasn't propagated to all instances.

Prevention: Run all migrations against a production-snapshot database in staging before prod. Require DBA or senior engineer approval for every migration. Use `dryRun` Flyway feature to validate SQL before execution.

---

45. Health check is passing but the service is returning wrong data after a config refresh. Why?

Cause: `@RefreshScope` beans were refreshed mid-request. Some beans used the old config, some the new, causing inconsistent state. Or the refresh event fired during a rolling deploy and different pods have different configs.

Fix: Use `@RefreshScope` carefully — only on beans that truly need live config. Test config refresh in staging. For rolling deploys, ensure all pods pick up the new config atomically by deploying them with the new ConfigMap version.

Prevention: Treat config changes as deployments — version them, test them, roll them out with the same care as code changes. Use Spring Cloud Config with Git-backed config and PR-based review for config changes.

---

46. An N+1 query problem is hammering the database in production. How do you detect and fix it?

Cause: Hibernate fetches a parent entity list, then fires one SELECT per child per parent — resulting in 1 + N queries instead of 1 JOIN.

Fix: Enable Hibernate statistics: `spring.jpa.properties.hibernate.generate_statistics=true`. Use `JOIN FETCH` in JPQL or `EntityGraph` to fetch associations in one query. For batch loading, use `@BatchSize` on the collection.

Prevention: Enable `spring.jpa.show-sql=true` and `hibernate.format_sql=true` in staging and count queries during integration tests. Use p6spy or datasource-proxy to assert max query count in tests.

---

47. Service fails on startup with `BeanCurrentlyInCreationException` — circular dependency. How do you resolve it?

Cause: Bean A needs Bean B in its constructor, and Bean B needs Bean A — Spring can't resolve this with constructor injection.

Fix: Refactor to break the circular dependency — usually a sign of poor separation of concerns. Extract a third service that both A and B depend on. As a last resort, use `@Lazy` on one injection point to defer initialization. Avoid `@Autowired` field injection just to paper over the circle.

Prevention: Use constructor injection everywhere — circular dependencies become compile-time obvious. Treat circular dependency detection as a design smell to fix, not work around.

---

48. Service is deployed but old users are hitting old API behavior. CDN or load balancer is serving stale responses. What do you check?

Cause: API responses are cached at the CDN (CloudFront, Akamai) or at a reverse proxy (NGINX) without cache invalidation after deploy.

Fix: Set proper `Cache-Control: no-store` or `Cache-Control: max-age=0, must-revalidate` headers on API responses. Perform a cache invalidation via CDN API on every deployment for affected paths. Verify the Load Balancer isn't doing response caching (it shouldn't for APIs).

Prevention: REST APIs should never be cached at the CDN level without explicit versioning strategy. Add `Cache-Control` headers as a default in your Spring Security or filter config. Test post-deploy behavior from outside the VPC to catch CDN caching.

---

49. How do you handle a thread dump when production is unresponsive but the process is still running?

Cause: Thread deadlock, thread pool exhaustion, or all threads are blocked waiting on an external resource (DB, network).

Fix: Take a thread dump without restarting: `jstack <pid> > threaddump.txt` or `kill -3 <pid>` (prints to stdout). In Kubernetes: `kubectl exec <pod> -- jstack 1`. Analyze with fastThread.io or manually look for `BLOCKED` and `WAITING` threads. Identify the lock owner. Restart the pod if needed but keep the dump for post-mortem.

Prevention: Add thread pool metrics via Micrometer. Set thread pool size limits and alert when active thread count approaches max. Use structured logging with thread name included in every log line.

---

50. Production incident — service is up but 30% of requests are failing silently. How do you run the post-mortem?

Cause: Could be anything — partial rollout of bad code, one unhealthy pod in the pool, a downstream service degraded but not fully down, a specific data pattern causing failures.

Fix (immediate): Check pod health individually — `kubectl exec` into each pod and curl the health endpoint. Check error rates per pod via load balancer access logs. Roll back the recent deployment if it correlates with the onset. Check Kafka lag for that time window. Check slow query log for DB anomalies.

Post-mortem process: Define the incident timeline. Identify contributing factors (code, config, infra, process). Write a blameless RCA. For every root cause, define a concrete action item: a code fix, a new alert, a process change. Track action items to completion. Share the RCA across teams.

Prevention: Define p99 error rate alerting at 1% threshold — catch 30% failures long before users report them. Use canary deployments so bad code only affects a small percentage of traffic before full rollout. Implement synthetic monitoring (scheduled probe requests) that alert immediately on business-path failures.

These 50 Q&As cover the most common production failure categories: memory, threading, database, Kafka, security, distributed systems, deployment, and observability. The pattern across all of them is the same — **detect early with metrics, design for failure with resilience patterns, and prevent with testing and code standards.**

6/2/2026, 5:43:01 PM